

Lowercasing

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import string
```

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
True
```

01. DATA LOWERCASING

```
def data_process(raw_dataset):
    raw_dataset = "Hello World! My Name Is Muhammad Saeed JaNgDa and I aM Learning AI ml course."
    lowercased_text = raw_dataset.lower()
    print(lowercased_text)
```

```
hello world! my name is muhammad saeed jangda and i am learning ai ml course.
```

02. Tokenization

```
#pip install nltk
```

```
tokens = word_tokenize(lowercased_text)
```

03. Stopwords

```
print(stopwords.words("english"))
```

```
['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren', "aren't", 'as', 'at'
```

```
#remove english stopwords function
def remove_stopwords(text, language):
    stop_words = set(stopwords.words(language))
    words = text.split()
    filtered_words = [word for word in words if word.lower() not in stop_words]
    print(language)
    print(filtered_words)
```

```
remove_stopwords(lowercased_text, 'english')
```

```
english
['hello', 'world!', 'name', 'muhammad', 'saeed', 'jangda', 'learning', 'ai', 'ml', 'course.']
```

QUESTION 02

Image Data Reduction

```
import numpy as np
```

```
def manual_pooling(feature_map):

    #Getting Dimension
    h, w = feature_map.shape

    #Other Dimension for 2*2 Pooling
```

```

new_h, new_w = h//2, w//2
new_feature_map = np.zeros((new_h, new_w))

for i in range(0, new_h):
    for j in range(0, new_w):
        slice_2x2 = feature_map[i*2:i*2+2, j*2:j*2+2]
        # Take the maximum value in that slice
        new_feature_map[i, j] = np.max(slice_2x2)

return new_feature_map

print("Pooled Map:\n", manual_pooling(sample_map))

```

```

Pooled Map:
[[8. 3.]
 [9. 7.]]

```

Question 03

```

import nltk

sentence = 'The quick brown fox jumps over the lazy dog'
words = nltk.word_tokenize(sentence)
pos_tags = nltk.pos_tag(words)

print("")
print("POS Tags\n", pos_tags, "\n")
nltk.download('tagsets_json')

for (word, tag) in pos_tags:
    print(nltk.help.upenn_tagset(tagpattern = tag))

```

```

POS Tags
[('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN'), ('jumps', 'VBZ'), ('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ')

DT: determiner
    all an another any both del each either every half la many much nary
    neither no some such that the them these this those
None
JJ: adjective or numeral, ordinal
    third ill-mannered pre-war regrettable oiled calamitous first separable
    ectoplasmic battery-powered participatory fourth still-to-be-named
    multilingual multi-disciplinary ...
None
NN: noun, common, singular or mass
    common-carrier cabbage knuckle-duster Casino afghan shed thermostat
    investment slide humour falloff slick wind hyena override subhumanity
    machinist ...
None
NN: noun, common, singular or mass
    common-carrier cabbage knuckle-duster Casino afghan shed thermostat
    investment slide humour falloff slick wind hyena override subhumanity
    machinist ...
None
VBZ: verb, present tense, 3rd person singular
    bases reconstructs marks mixes displeases seals carps weaves snatches
    slumps stretches authorizes smolders pictures emerges stockpiles
    seduces fizzes uses bolsters slaps speaks pleads ...
None
IN: preposition or conjunction, subordinating
    astride among upon whether out inside pro despite on by throughout
    below within for towards near behind atop around if like until below
    next into if beside ...
None
DT: determiner
    all an another any both del each either every half la many much nary
    neither no some such that the them these this those
None
JJ: adjective or numeral, ordinal
    third ill-mannered pre-war regrettable oiled calamitous first separable
    ectoplasmic battery-powered participatory fourth still-to-be-named
    multilingual multi-disciplinary ...
None
NN: noun, common, singular or mass
    common-carrier cabbage knuckle-duster Casino afghan shed thermostat
    investment slide humour falloff slick wind hyena override subhumanity
    machinist ...
None

```

```
[nltk_data] Downloading package tagsets_json to /root/nltk_data...
[nltk_data] Package tagsets_json is already up-to-date!
```

```
pip install svgling
```

```
Requirement already satisfied: svgling in /usr/local/lib/python3.12/dist-packages (0.5.0)
Requirement already satisfied: svgwrite in /usr/local/lib/python3.12/dist-packages (from svgling) (1.4.3)
```

```
nltk.download('maxent_ne_chunker')
nltk.download('punk_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('words')
```

```
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data] /root/nltk_data...
[nltk_data] Package maxent_ne_chunker is already up-to-date!
[nltk_data] Error loading punk_tab: Package 'punk_tab' not found in
[nltk_data] index
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data] Package words is already up-to-date!
True
```

```
nltk.download('maxent_ne_chunker_tab')
```

```
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data] /root/nltk_data...
[nltk_data] Package maxent_ne_chunker_tab is already up-to-date!
True
```

```
from nltk.tree import Tree
from nltk.chunk import ne_chunk
```

```
words = nltk.word_tokenize(sentence)
pos_tags = nltk.pos_tag(words)

chunks = ne_chunk(pos_tags)

reg_exp = "NP: {<DT>?<JJ>*<NN>}"
parser = nltk.RegexpParser(reg_exp)
result = parser.parse(chunks)
result
```

